Sistema de Scheduling Distribuido con Tolerancia a Fallos

Documento Técnico de Arquitectura y Diseño

Fase 2: Proyecto sistemas complejos

Sistemas Distribuidos

Juan Camlio Gallardo Sánchez

Mariana Ruge Vargas

Universidad Sergio Arboleda

21 de Abril del 2026

Índice

El presente documento se organiza en las siguientes secciones:

1. Introducción y objetivos
2. Arquitectura del sistema
3. Módulos y dependencias
4. Modelo de datos en Redis
5. Algoritmo de scheduling
6. Flujos de operación
7. Tolerancia a fallos
8. Memoria distribuida
9. Primitivas de sincronización
10. Sistema de métricas
11. Diagrama de casos de uso
12. Diagrama de clases
13. Guía de ejecución
14. Conclusiones

1. Introducción y objetivos

1.1 Contexto

Los sistemas de scheduling distribuidos constituyen la columna vertebral de la computación moderna en entornos de alta disponibilidad. Su función consiste en coordinar la asignación de trabajo (tareas) entre múltiples nodos de cómputo, garantizando el balanceo de carga, el aprovechamiento eficiente de recursos y la continuidad del servicio ante fallos parciales. Un scheduler distribuido debe resolver simultáneamente problemas clásicos como la selección óptima de nodo, la consistencia de estado compartido, la detección fiable de fallos y la reconfiguración dinámica de la topología.

Este proyecto implementa un prototipo funcional de scheduler distribuido en Python, empleando Redis como message broker y almacén de estado compartido. El sistema se ejecuta sobre tres procesos independientes que simulan tres nodos de un cluster: un nodo coordinador denominado scheduler y dos nodos trabajadores denominados worker-1 y worker-2. La arquitectura está diseñada para ilustrar los principales desafíos del cómputo distribuido en un entorno reproducible y de complejidad acotada.

1.2 Objetivos

Los objetivos específicos del proyecto son los siguientes:

- Diseñar e implementar un algoritmo de selección de nodo óptimo basado en reputación, carga y estado de salud.
- Desarrollar un mecanismo de detección de fallos basado en heartbeats con transiciones de estado ALIVE, SUSPECTED y FAILED.
- Garantizar la reconfiguración automática del cluster ante la caída de un nodo, reasignando atómicamente las tareas pendientes.
- Construir una abstracción de memoria distribuida con replicación, versionado y tolerancia a fallos mediante fallback.
- Proveer primitivas de sincronización distribuida: mutex exclusivo, barrera de participantes y reloj lógico de Lamport.
- Instrumentar el sistema con un subsistema de métricas que registre eventos, latencias y contadores por nodo.

1.3 Alcance

El documento describe la arquitectura completa del sistema, detallando los siete módulos que lo componen, las estructuras de datos persistidas en Redis, los algoritmos de decisión y los flujos de operación en régimen normal y ante fallos. Se incluyen diagramas formales en notación Mermaid que acompañan cada sección relevante. El documento no aborda aspectos de seguridad perimetral, autenticación ni despliegue en producción, los cuales quedan fuera del alcance académico de este prototipo.

2. Arquitectura del sistema

2.1 Visión general

La arquitectura adopta un modelo híbrido coordinador-trabajadores con estado compartido. El nodo scheduler asume la responsabilidad exclusiva de asignar tareas, mientras que los nodos worker-1 y worker-2 consumen tareas de sus colas dedicadas y ejecutan la lógica de negocio. Todos los nodos mantienen en Redis una estructura de heartbeat y contribuyen a la memoria compartida, la detección de fallos y el sistema de métricas. Redis cumple así un triple rol: broker de mensajes mediante Pub/Sub, almacén clave-valor para estado replicado y motor de ejecución atómica mediante scripts Lua.

La comunicación entre nodos es asíncrona y se apoya en dos mecanismos complementarios. Por un lado, las colas implementadas con listas (LPUSH/RPOP) permiten el encolado durable de tareas asignadas. Por otro lado, los canales Pub/Sub permiten notificaciones en tiempo real de eventos transversales como cambios en la memoria distribuida, reconfiguraciones de topología y mensajes directos entre nodos.

2.2 Diagrama de arquitectura general

El siguiente diagrama presenta los tres nodos del sistema y sus flujos principales de comunicación a través de Redis:

Relaciones entre Nodos

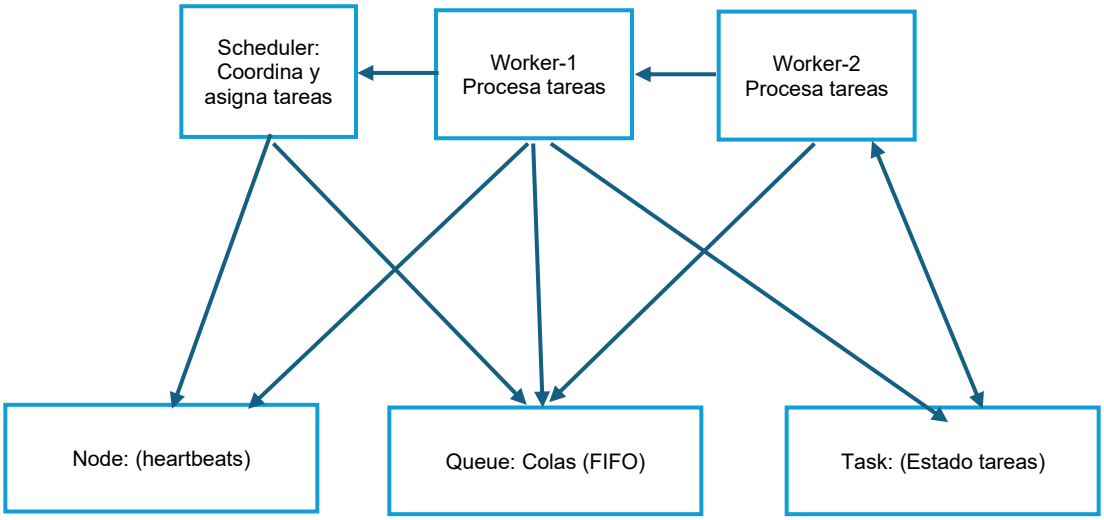

Figura 1. Arquitectura general del sistema: tres nodos coordinados a través de Redis.

2.3 Roles y responsabilidades

Cada nodo del sistema ejecuta el mismo código base pero con un rol diferenciado mediante un argumento de línea de comandos. El rol define qué hilos se activan y qué comandos están disponibles en el CLI interactivo:

Rol	Responsabilidades	Hilos activos
scheduler	Asignación de tareas, administración del cluster, simulación de fallos	Heartbeat, FailureDetector, ReconfigurationManager, LogicalClock, Metrics
worker-1 / worker-2	Consumo y ejecución de tareas, reporte de resultados	Heartbeat, consumo de cola, LogicalClock, Metrics

3. Módulos y dependencias

3.1 Organización del código

El sistema se estructura en siete módulos Python autocontenidos que siguen el principio de responsabilidad única. Cada módulo encapsula un aspecto funcional del sistema y expone una interfaz pública bien definida. La siguiente tabla resume cada módulo:

Módulo	Responsabilidad
config.py	Configuración centralizada del sistema mediante la dataclass SimConfig (intervalos, umbrales, parámetros de carga).
synchronization.py	Primitivas de coordinación: DistributedLock (SETNX+UUID+Lua), DistributedBarrier (SET+polling) y LogicalClock (Lamport con Pub/Sub).
distributed_memory.py	Clase DistributedMemory: almacenamiento replicado con versionado, fallback a réplicas, cache local y notificaciones por Pub/Sub.
metrics.py	Clase Metrics: contadores por nodo (assigned/completed/failed), registro de latencias, stream de eventos y exportación a JSON/texto.
scheduler.py	Clase DistributedScheduler: selección de nodo óptimo, heartbeat, carga simulada cíclica, decaimiento de reputación y scripts Lua atómicos.
fault_tolerance.py	FailureDetector (ALIVE→SUSPECTED→FAILED), ReconfigurationManager (reasignación atómica vía Lua + broadcast) y utilidad simulate_crash().
main.py	Entry point con argparse, CLI interactivo diferenciado por rol y shutdown limpio ante señales del sistema operativo.

3.2 Diagrama de componentes

El siguiente diagrama muestra las dependencias entre módulos. Las flechas indican relación de importación: A → B significa que el módulo A importa B. La organización garantiza un grafo acíclico donde main.py se sitúa en la cima y config.py en la base.

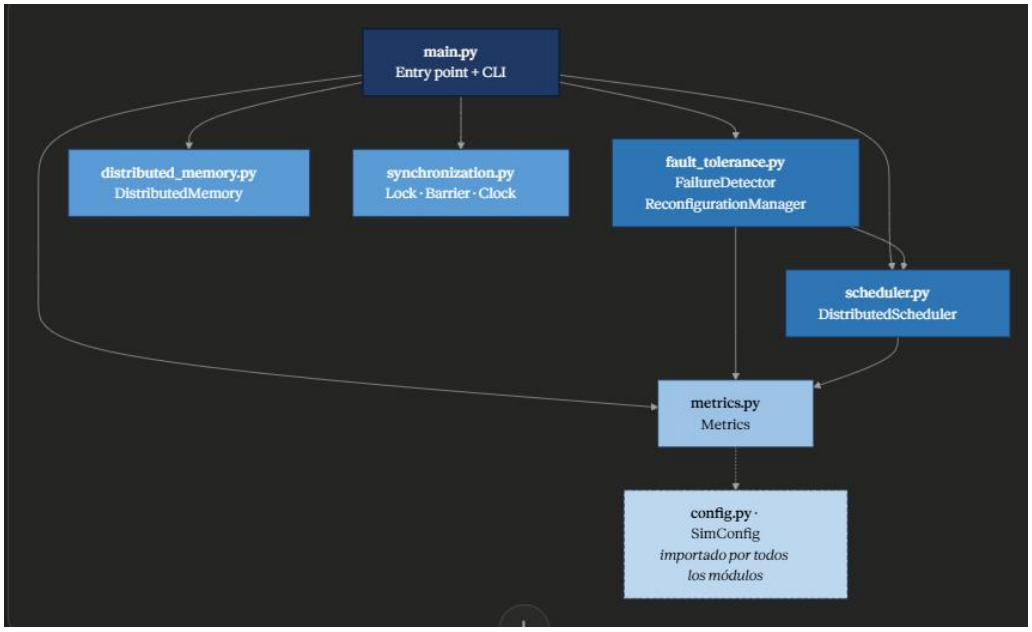

Figura 2. Grafo de dependencias entre los siete módulos del sistema.

4. Modelo de datos en Redis

Todo el estado distribuido del sistema se persiste en Redis mediante un conjunto de claves con convenciones de nomenclatura estrictas. La siguiente tabla documenta cada categoría de clave, su tipo de dato en Redis y los campos que contiene:

Clave	Tipo	Contenido / campos
node:{id}	Hash	last_heartbeat, available, reputation, current_load, tasks_in_queue
task:{uuid}	Hash	id, assigned_to, data, status, created_at, completed_at, result
queue:{id}	List	IDs de tareas pendientes (FIFO: LPUSH + RPOP)
lock:{name}	String	UUID token del holder actual del mutex distribuido
barrier:{name}	Set	node_ids de los participantes que han llegado a la barrera
lclock:{id}	String	Timestamp Lamport del nodo (contador monótonamente creciente)
dmem:{key}	Hash	data, version, updated_by, updated_at
dmem:{key}:replica:N	Hash	Copia del hash principal (una clave por réplica)
metrics:node:{id}	Hash	Contadores por nodo: assigned, completed, failed
metrics:events	List	Stream de eventos serializados como JSON
metrics:latency:{id}	List	Últimas 100 latencias medidas en milisegundos

4.1 Canales Pub/Sub

Adicionalmente, el sistema utiliza tres canales Pub/Sub para propagación asíncrona de eventos:

- **dmem:changes** — notifica cambios en la memoria distribuida para invalidar cachés locales y disparar reacciones.
- **cluster:topology** — difunde cambios en la topología del cluster (nodos nuevos, caídas, reconfiguraciones).
- **msg:{node_id}** — canal dedicado por nodo para mensajes directos punto a punto, utilizado por LogicalClock para intercambio de timestamps Lamport.

5. Algoritmo de scheduling

5.1 Fórmula de scoring

La selección del nodo óptimo para ejecutar una tarea se basa en una función de puntuación compuesta que combina cuatro factores: reputación histórica del nodo, carga instantánea, estado de salud y saturación de la cola. La fórmula es la siguiente:

$$\text{score} = (\text{reputación} \times 0.4) + ((100 - \text{carga}) \times 0.5) + (\text{nodo_vivo} \ ? \ 10 : 0) - \text{penalización_cola}$$

Cada componente de la fórmula tiene el siguiente significado y rango de valores:

Reputación (0–100)

- Valor inicial: 70 puntos al registrar un nodo.
- Incremento: +5 puntos por cada tarea completada con éxito.
- Penalización: −10 puntos por cada tarea fallida.
- Decaimiento: 0.5 puntos por hora, garantizando que reputaciones antiguas no dominen indefinidamente.

Carga simulada (0–100)

- Patrón sinusoidal por nodo para emular ciclos de ocupación realistas.
- Fórmula: carga = base + amplitud × sin(2π × t / período) + ruido
- Cada nodo posee parámetros distintos de base, amplitud y período para introducir heterogeneidad.

Penalización de cola

- 5 puntos de penalización por cada tarea encolada.
- Penalización máxima: 30 puntos (equivalente a 6 tareas en cola o más).
- Evita que un nodo saturado reciba más trabajo aunque su reputación sea alta.

Bonus por salud

- Se suman 10 puntos si el nodo está vivo (heartbeat reciente); 0 en caso contrario.

5.2 Diagrama de flujo del scoring

El siguiente diagrama ilustra el proceso completo desde la llegada de la solicitud de scheduling hasta la selección del nodo ganador:

Figura 7. Flujo de decisión del algoritmo de scoring y asignación.

6. Flujos de operación

6.1 Asignación y ejecución de tarea

El flujo completo desde que el administrador ingresa el comando task en el CLI del scheduler hasta que un worker completa su ejecución involucra múltiples interacciones con Redis. El siguiente diagrama de secuencia detalla cada paso:

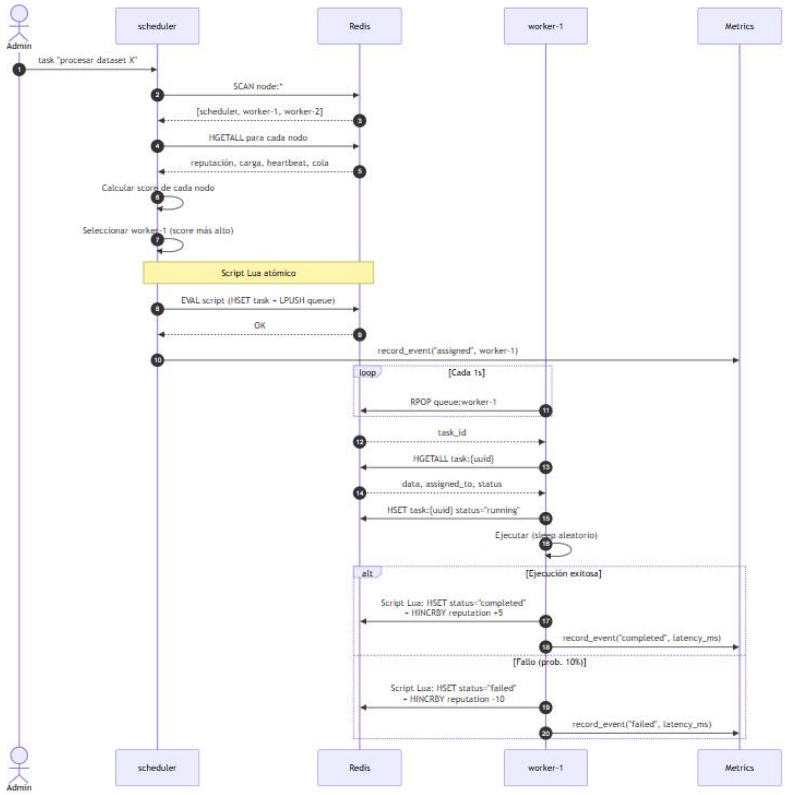

Figura 3. Secuencia de asignación y ejecución de una tarea.

6.2 Observaciones sobre atomicidad

Los pasos críticos del flujo (asignación y actualización de estado tras ejecución) se implementan mediante scripts Lua que Redis ejecuta de manera atómica y single-threaded. Esto elimina las condiciones de carrera entre la escritura del hash de tarea y el encolado, así como entre la actualización de estado y el ajuste de reputación. Si el scheduler cayese entre un HSET y un LPUSH sin atomicidad, una tarea podría quedar huérfana en el almacén sin estar realmente asignada.

7. Tolerancia a fallos

7.1 Detección de fallos

La detección de fallos se realiza mediante un esquema de heartbeats con dos umbrales sucesivos que permiten distinguir entre nodos temporalmente lentos (SUSPECTED) y nodos efectivamente caídos (FAILED). Esta gradación reduce los falsos positivos en redes con latencia variable.

Cada nodo publica su heartbeat cada 2 segundos en la clave node:{id} con un TTL de 10 segundos. El FailureDetector, que reside en el scheduler, recorre todos los nodos cada 3 segundos y aplica las siguientes reglas de transición:

7.2 Diagrama de estados del nodo

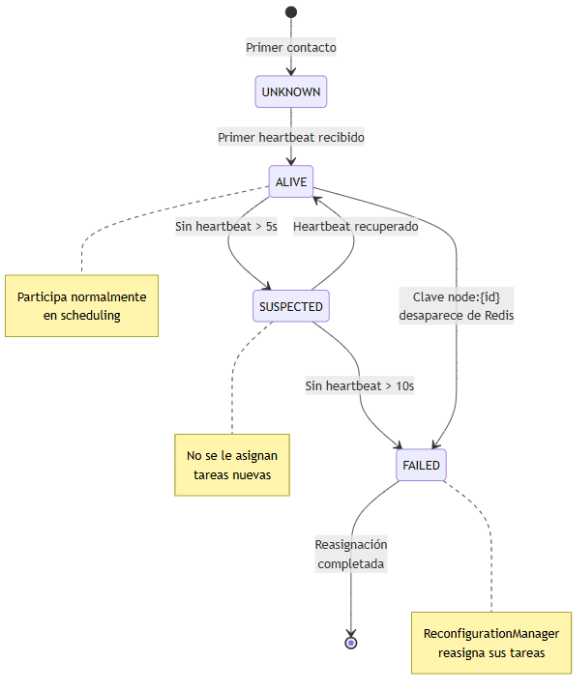

Figura 5. Diagrama de estados de un nodo según el FailureDetector.

7.3 Reconfiguración del cluster

Cuando un nodo transiciona al estado FAILED, el ReconfigurationManager se activa automáticamente. Su misión consiste en reasignar todas las tareas pendientes del nodo caído a un nodo vivo, garantizando que ninguna tarea quede perdida en una cola inaccesible. La operación se implementa mediante un script Lua que transfiere atómicamente el contenido de la cola y actualiza el status de cada tarea a reassigned.

7.4 Diagrama de secuencia: fallo y reconfiguración

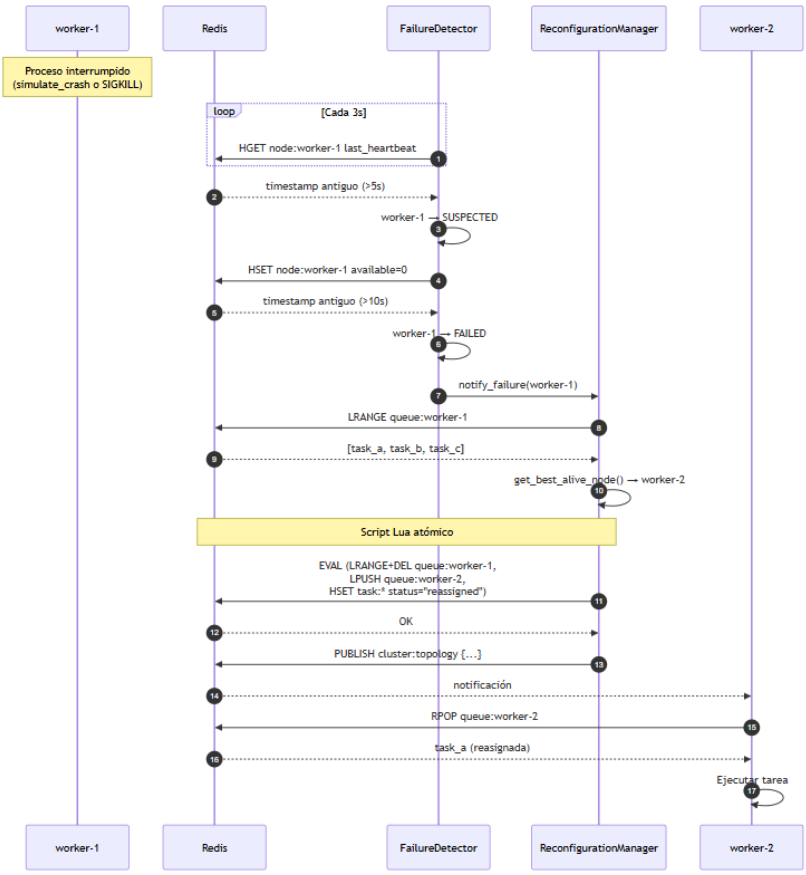

Figura 4. Secuencia de detección de fallo y reconfiguración automática.

8. Memoria distribuida

8.1 Diseño

La clase DistributedMemory ofrece una abstracción clave-valor replicada y versionada sobre Redis. Su propósito es permitir que los nodos del cluster compartan datos con garantías de consistencia eventual y disponibilidad incluso si la clave primaria se pierde. Los tres pilares del diseño son el versionado atómico, la replicación explícita y el fallback transparente.

8.1.1 Operaciones

- **store(key, value)** — escribe un hash con los campos data, version, updated_by, updated_at. La versión se incrementa atómicamente con HINCRBY y se publica una notificación en dmem:changes.
- **replicate(key, N)** — crea N copias en claves dmem:{key}:replica:1 hasta dmem:{key}:replica:N.
- **get(key)** — lee del primario; si falla, prueba cada réplica; si todas fallan, devuelve el valor desde la cache local del nodo.
- **invalidate(key)** — borra el primario, todas las réplicas y notifica la invalidación.

8.2 Diagrama de la memoria distribuida

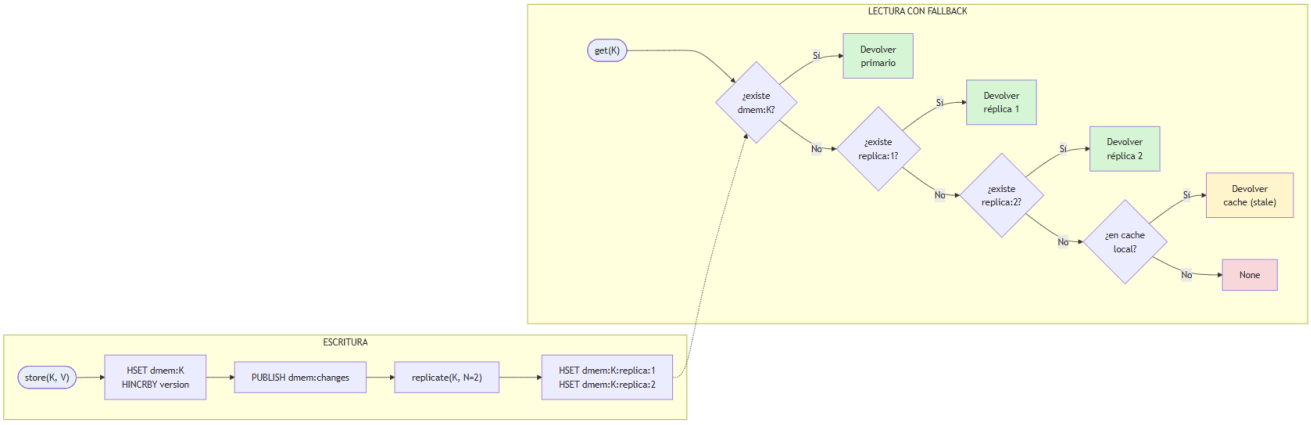

Figura 8. Flujo de escritura con replicación y lectura con fallback en cascada.

8.3 Notificación por Pub/Sub

Cada escritura publica un mensaje en el canal dmem:changes conteniendo la clave afectada, la nueva versión y el nodo autor. Los nodos suscritos reaccionan invalidando sus cachés locales para esa clave, garantizando que la siguiente lectura obtenga el valor más reciente. Este mecanismo evita el polling continuo y reduce la carga sobre Redis.

10. Sistema de métricas

10.1 Objetivo

El subsistema de métricas instrumenta todas las operaciones relevantes del sistema para permitir observabilidad en tiempo real y análisis post-mortem. La clase Metrics encapsula contadores, histogramas de latencia y un stream de eventos, todos persistidos en Redis para ser accesibles desde cualquier nodo.

10.2 Categorías de métricas

10.2.1 Contadores por nodo

Almacenados en metrics:node:{id} como un hash con tres campos: assigned (tareas recibidas), completed (tareas terminadas con éxito) y failed (tareas con error). Los incrementos se realizan con HINCRBY, operación atómica en Redis.

10.2.2 Latencias

Para cada nodo se mantiene una lista metrics:latency:{id} con las últimas 100 mediciones de tiempo de ejecución en milisegundos. Se implementa mediante LPUSH seguido de LTRIM 0 99 para preservar una ventana deslizante de tamaño fijo.

10.2.3 Stream de eventos

La lista metrics:events actúa como un log centralizado donde cada entrada es un objeto JSON con al menos los campos timestamp, event_type, node_id y payload. Este stream puede consumirse para reconstruir la línea temporal completa del sistema.

10.3 Exportación

La clase Metrics expone dos métodos de exportación: export_json() produce un snapshot completo del estado actual en formato JSON, mientras que export_text() genera un reporte tabulado legible por humanos, adecuado para incluir en informes o presentaciones.

11. Conclusiones

- El sistema descrito en este documento implementa un scheduler distribuido completo que integra los conceptos fundamentales del cómputo distribuido en un prototipo acotado pero funcional. La arquitectura elegida, centrada en Redis como fuente de verdad compartida, permite ilustrar con claridad los problemas clásicos del área: selección óptima de recursos, detección de fallos, reconfiguración dinámica, sincronización distribuida y consistencia de estado replicado.
- Entre los aportes más relevantes del proyecto cabe destacar el algoritmo de scoring compuesto, que balancea múltiples factores mediante una fórmula ponderada y resulta fácilmente extensible. La combinación de reputación histórica con carga instantánea y penalización por saturación proporciona un comportamiento estable incluso ante cargas heterogéneas. Asimismo, el uso consistente de scripts Lua para operaciones críticas garantiza la atomicidad necesaria en puntos donde las condiciones de carrera podrían comprometer la consistencia.
- La separación entre FailureDetector y ReconfigurationManager sigue el principio de responsabilidad única y facilita el razonamiento sobre el comportamiento del sistema ante fallos. El esquema de dos umbrales (SUSPECTED y FAILED) permite tolerar perturbaciones transitorias sin disparar reconfiguraciones innecesarias, un compromiso habitual en sistemas reales. La memoria distribuida con replicación y fallback en cascada demuestra cómo construir abstracciones de disponibilidad elevada sobre primitivas simples.